

Extra Capstone: City Disruption Memory

An open-source, live city-data product with AI

Question: What reported infrastructure issues and transit disruptions overlapped around a stop or corridor, what changed during the week, and what evidence did the system have at the time?

User: a local reporter, community researcher, or resident investigating a specific area. The product offers an evidence timeline, not safety advice or guaranteed travel directions.

Working city: Chicago, because official 311, transit-alert, and transit-geography sources are available. The project can later support another city through adapters. Build one city first.

Deliverable in this folder: a build specification and learning schedule. The application, collectors, and dataset are not implemented by this document.

Commitment	Schedule
Core AI course	Keep D01–D60, 12 weeks, 360 hours, and the existing support capstone
Extra city-data capstone	Proposed D61–D75, three additional five-day weeks
Daily study time	Six hours, with weekends off
Extra study hours	90
Combined course + extension	75 study days, 15 weeks, 450 hours

The extension is additional work, not a hidden task squeezed into the original 60 days. Its schedule assumes you can reuse the existing Airflow, FastAPI, evaluation, logging, and structured-output components. If source access, evidence collection, or a completion gate takes longer, extend the finish date instead of replacing real evidence with synthetic results.

1. What makes this your project

You will not download a finished city-analysis dataset or clone a tutorial pipeline. You will collect official operational records, retain source changes, construct the stop/corridor overlap dataset, label real examples, and publish your methodology.

The original contribution is the **derived disruption history and its evidence model**. The city still owns its source records; collection does not transfer their copyright or remove their terms.

Defensible differentiation

Existing projects already analyze Chicago bus reliability:

- [CTA StopWatch](#) combines collected bus locations, schedules, and geography to measure service reliability.
- [Bus Pending](#) collects bus positions and analyzes delay patterns with schedules and demographic information.

Do not pitch another bus-delay dashboard as new. This capstone targets a different question: **cross-source, stop-level disruption evidence with corrections and historical knowledge cutoffs**. Both temporal history and spatial joins have established precedents; the proposed contribution is their application to this particular evidence product and your collected data. A limited search does not establish global uniqueness.

Your portfolio claim after completing the project can be: “I built an open-source pipeline that preserves Chicago transit-alert and infrastructure-report changes, computes reproducible local overlaps, and produces source-backed AI explanations that respect historical knowledge cutoffs.” Do not claim measured performance or originality beyond what your comparison and results establish.

2. Source contracts and real-data boundaries

Source	Official entry point	Use	Important limit
Chicago 311 service requests	Dataset · API sample · Metadata	Report type, location, administrative status, source timestamps	A complaint is a report, not a verified physical obstruction. Status Completed does not prove the street is clear.
CTA Customer Alerts	Official API documentation	Planned/unplanned service events, affected services, descriptive text	Preserve structured fields first; AI extracts only facts the text supports. Capture a full successful feed before inferring absence.
CTA GTFS	Official GTFS documentation · Feed download	Stops, route IDs, stop-to-route membership, schedule/geography versions	Scheduled service is not actual service. Retain feed versions so current geography does not rewrite history.

Source verification performed while planning: a live two-row 311 response contained sr_number, sr_type, status, created_date, last_modified_date, and closed_date; only one sampled row contained coordinates. The official metadata warns that 311 INFORMATION ONLY CALL locations often identify the 311 center rather than the caller's issue. Exclude that category from spatial analysis. CTA documentation identifies an alerts API and GTFS distribution. CTA payload/GTFS ingestion has not been executed here.

Terms: Chicago's metadata uses SEE_TERMS_OF_USE, not a blanket open-source data license. CTA links a [Developer License Agreement and Terms of Use](#). Review these before collection and redistribution. Do not claim your code license covers source data or assume every snapshot can be published. Document attribution, retention, and permitted sharing; publish derived aggregates or acquisition scripts where raw redistribution is restricted.

Bound the first build

Choose one corridor crossing no more than three adjoining community areas and at most 30 stops after inspecting GTFS. Start with three directly relevant 311 types, such as street-light, traffic-signal, and pothole reports; verify their current exact type values. Store other categories outside the analytical model only if needed for source completeness.

Do not add crime data, individual rider tracking, route optimization, real-time vehicle tracking, weather, or construction permits to this capstone. Those create separate questions and source contracts. Add a source later only if it changes a defined user decision.

Collect your own observations

Start collectors on D62 and retain at least seven calendar days of actual observations by the final demo. Unattended jobs may run on weekends; you do not need weekend study sessions. Keep collectors active after the build to develop a longer history.

A starting cadence, subject to each source's current limits, is CTA alerts every 15 minutes, scoped 311 changes every two hours, and a daily GTFS version check. Log failed polls and rate-limit responses. Reduce cadence if required; freshness metrics must use the configured cadence.

Backfill up to 30 days of relevant 311 records for context. **Backfilled current records do not establish what the API showed in the past.** Your historical “known at” feature starts with your first successful collection. Do not manufacture historical CTA versions or describe today's version as yesterday's knowledge.

If the collection window contains no alert corrections or genuine cross-source overlaps, show the honest empty result and keep collecting. Small synthetic fixtures are allowed for tests, but not as the product dataset or as proof of a real city incident. Record a real correction/overlap only when one is observed; its occurrence is not guaranteed by the schedule.

3. Product behavior

The UI needs four views:

1. **Area/stop timeline:** source events and relevant nearby reports, with source timestamps and freshness.
2. **Changes:** new, changed, and no-longer-observed records, showing before/after source versions.
3. **Evidence drawer:** source ID/URL, snapshot hash, captured time, source-effective time, extraction span, matching rule, and geography version.
4. **Explanation:** a short AI account of the selected evidence, with citations, ambiguity, and gaps.

Provide deterministic filters for area/stop, date range, and knowledge cutoff. Natural-language chat is not required. A “What changed?” button is enough if it serves the question.

Separate the two timelines

- **Source-effective time:** when the provider says an alert or status applies. Missing ends remain unknown, not an invented resolution.
- **Observation/knowledge time:** when your system captured a source version. For AI-derived facts, knowledge time is the later of capture time and the first persisted validated extraction. Preserve human-review time as well.

For a historical view, use only raw versions, geography, and derived facts available by the requested cutoff. A model run performed today cannot appear in a reconstructed answer supposedly generated yesterday. Offer a separate, labelled retrospective reanalysis if you process old evidence with a new model.

The 311 created_date/closed_date interval can indicate the lifetime of the **administrative report**. It must not be labelled the physical hazard's duration. last_modified_date helps ingestion; it does not reconstruct every prior status transition.

4. Data engineering architecture

Use Python collectors, Airflow, dbt, PostgreSQL + PostGIS, FastAPI, Streamlit, and Docker Compose. Use immutable compressed JSON/XML snapshots and the original GTFS archive on a local persistent volume; generate Parquet exports from normalized records when useful. Use an open-weight local model through Ollama for extraction/explanation so the demonstrated stack has no mandatory paid model dependency. Record the chosen model's license and hardware needs.

Do not add Kafka, Spark, Kubernetes, a graph database, a vector store, or several agent frameworks just to increase the stack. The project requires scheduled collection, relational history, spatial matching, and evidence retrieval. PostGIS and SQL are sufficient for the bounded workload.

Flow: collectors → immutable raw evidence → validated source versions → versioned geography and extracted facts → SQL/PostGIS overlaps and change marts → API/UI → cited AI explanation. Failed data goes to quarantine with a reason; failed models leave the deterministic timeline usable.

Stored entity	Required fields / behavior
collection_runs	Source, query scope, start/end, outcome, pages, count, checkpoint, completeness, failure reason
raw_snapshots	Snapshot ID, source URL/query without secrets, capture UTC, response metadata, SHA-256, storage path, collection-run ID
source_versions	Source + record ID, canonical payload hash, snapshot reference, source timestamps, observed interval; append a changed version rather than overwrite it
geography_versions	GTFS feed hash, captured/published metadata, stop/route geometry and membership; valid service dates where available
extracted_facts	Source-version ID, explicit extracted fields, evidence spans, parser/model/prompt version, extraction UTC, validation/review status
disruption_overlaps	Both source-version IDs, stop/corridor ID, geography version, spatial rule, temporal rule, distance, match certainty, derivation UTC
explanations	Request filters/cutoff, exact input evidence IDs, output citations, model/prompt version, created UTC, validation outcome, runtime

Incremental collection and replay

For 311, page by a stable order using source modification time plus record ID; use an overlapping lookback to catch equal timestamps and delayed updates. Persist a completed checkpoint only after all pages and raw evidence are durable. Deduplicate by source/record/version. Inspect the actual API contract before choosing pagination syntax.

For alerts, capture the complete feed and retain record-level versions. A record absent from a partial or failed poll is **unknown**, not resolved. A successful complete feed can establish “no longer observed”; it still cannot prove physical resolution. Run bounded reconciliations so missed updates and source deletions become explicit states.

An identical payload should not produce another business version, but each poll still contributes collection/freshness evidence. Replaying stored snapshots must recreate the same normalized facts and marts when code/config/model outputs are fixed. Persist AI outputs; byte-identical reruns of a nondeterministic model are not a reproducibility guarantee.

Spatial and temporal matching

Use provider stop/route IDs when present, plus GTFS membership from the correct version. Parse named intersections/segments only when the alert supports them. Route-wide alerts belong to a route-wide layer; do not pretend they identify one street corner. Quarantine ambiguous locations rather than choosing a coordinate through LLM guesswork.

Use ST_DWithin on geography values in meters for a configurable candidate radius, initially 300 m. This is proximity, not a walking distance or proof of impact. Check sensitivity at 150/300/500 m on labelled examples before finalizing the rule.

Calculate time overlap in SQL from documented source intervals. Treat open-ended administrative reports as open-ended **reports**, with age visible. Do not merge two records into one incident just because their locations and times overlap. Label the output “co-occurring reports/alerts”; retain separate IDs and uncertainty.

Normalize timestamps to UTC while displaying America/Chicago. Preserve original source strings; resolve naive timestamps using the documented source timezone. Quarantine ambiguous daylight-saving times rather than silently selecting an offset.

5. AI that earns its place

AI has two bounded jobs:

- **Extract:** convert alert prose into typed fields such as place mentions, affected service, effective dates, and stated change reason. Require supporting text spans; prefer authoritative structured fields over generated replacements. Unknown fields remain null.
- **Explain:** narrate the SQL-selected overlap/change evidence. Require citations to the provided evidence IDs, and separate reported facts, inferences, and missing evidence.

The model does not calculate spatial distances, decide chronology, infer physical resolution from 311 closure, issue safety guarantees, or invent a causal account of a delay. Treat source text as untrusted data; it cannot authorize tool calls. Model failure should return a labelled unavailable explanation alongside the working evidence timeline.

Prove added value

Create 40 hand-labelled examples from your collected source material: 20 development and 20 held-out. Split by source event/record, not random versions of the same event, to avoid near-duplicate leakage. Include ambiguous geography, missing dates, unrelated nearby reports, corrections, and unsupported questions when the source collection provides them. Keep any artificial edge fixtures in a separate test set.

Compare rules/structured-fields-only extraction with rules + the local model. Measure field correctness, unsupported extraction rate, unresolved/abstain rate, citation support, latency, and inference resource use. Count correct abstentions rather than rewarding confident guesses. If the model adds no benefit on your sample, report that result and keep it out of authoritative matching.

For explanations, compare a deterministic evidence summary with the AI version using the same facts. Review whether AI improves comprehensibility without introducing unsupported claims. Freeze held-out cases before prompt tuning; report sample size and limitations without forcing a winner.

6. Three-week build schedule

Each row uses **1.5 h learn/design + 3 h build + 1 h prove + 0.5 h wrap = 6 h**. Friday includes repair, not additional scope.

Week 13: Acquire evidence and construct history

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D61 Mon	Review source terms, schemas, and existing transit projects; define the user question	Choose corridor/stops/categories; create source contracts, code license, and a tiny real-source acquisition spike	Confirm usable 311, alerts, and GTFS inputs under their terms. Record missing fields, source restrictions, and source-to-product mapping.
[] D62 Tue	Study checkpoints, retries, and immutable landing	Build scoped collectors and raw snapshot/run manifests; start scheduled collection immediately	Interrupt and rerun ingestion; verify checkpoint safety, hash integrity, and no secret-bearing query logs.
[] D63 Wed	Study PostGIS and versioned GTFS joins	Load GTFS stop/route membership and selected 311 categories; quarantine missing/invalid locations	Inspect 10 locations; reject 311 information-only calls; confirm distance units and preserve geography hash.
[] D64 Thu	Study observation time versus effective time	Implement source-version history, unchanged-payload deduplication, and coverage gaps	Replay duplicate, changed, late, and absent records using separate test fixtures; compare resulting state to expected history.
[] D65 Fri	Review source cadence and collection health	Finish incremental Airflow runs and raw replay; repair acquisition/history blockers	Rebuild normalized source history from stored evidence. Friday gate: each fact traces to a captured source and failures remain visible.

Week 14: Compute overlaps and constrain AI

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D66 Mon	Study temporal intervals and spatial candidate rules	Build dbt models for stop/corridor report-alert candidates; retain route-wide and unresolved matches separately	Verify distance/interval boundaries and demonstrate nearby-but-unrelated evidence. No inferred causal incident merge.
[] D67 Tue	Review structured output, spans, and source precedence	Add local-model extraction of alert prose with schema validation and ambiguous-location review	Label development examples from actual collection; reject unsupported dates/locations and handle model outage without losing data.
[] D68 Wed	Study historical cutoffs and derived-fact availability	Implement historical evidence queries and change summaries with pinned geography/extraction versions	Prove later corrections/extractions cannot leak into an earlier cutoff. Label retrospective reanalysis as distinct.
[] D69 Thu	Review evidence-grounded explanation patterns	Build FastAPI evidence/change endpoints and an explanation action over selected facts	Ask supported, ambiguous, and unsupported questions. Check each generated factual claim against its cited source.
[] D70 Fri	Review quality fixtures and comparison design	Freeze held-out examples; finish extraction baseline and evidence-summary comparison; repair blockers	Run evaluation and save honest scores. Friday gate: reproducible matching plus an AI baseline comparison, not just a successful chat.

Week 15: Product, reliability, and open-source release package

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D71 Mon	Review map/timeline interaction and uncertainty labels	Build Streamlit area/stop filters, dual-time timeline, change diff, evidence drawer, and freshness display	Walk through current versus historical views. Ensure old/missing source evidence never appears as a current all-clear.
[] D72 Tue	Review backfill, deletion reconciliation, and schema drift	Add bounded reconciliation, quarantine handling, and collector recovery; keep AI tools read-only	Exercise partial feed, HTTP error, schema change, lost coordinates, DST ambiguity, and model failure with isolated test fixtures.
[] D73 Wed	Review held-out evaluation and report limits	Run frozen real-source extraction/explanation evaluation; finalize the derived dataset/data dictionary and provenance	Inspect held-out errors, source gaps, match-radius sensitivity, runtime, and model resource usage. Distinguish test fixtures from collected evidence.
[] D74 Thu	Review source/code licensing and clean deployment	Finish Compose startup, acquisition/replay instructions, dependency pinning, attribution, runbook, and case study	Follow the README from a clean database and replay permitted evidence. Check that acquisition, rebuild, and review steps actually work.
[] D75 Fri	Review the contribution against comparable projects	Repair final blockers; finish demo recording, architecture diagram, source-contract index, and release-ready repository	Demo actual source evidence, historical cutoff, ambiguity, and failure recovery. Show empty findings when appropriate; do not manufacture a city incident.

“Release-ready” means the repository is prepared for publication; creating a public repository or publishing source snapshots requires your explicit approval and license review. No external publishing is authorized by this plan.

7. Completion criteria

- [] A bounded Chicago corridor/stop set and documented analytical question, not an unscoped city dashboard.
- [] Working collectors for permitted 311, CTA alerts, and GTFS sources, with at least seven days of actual observations and visible collection gaps.
- [] Immutable evidence manifests, versioned source records/geography, and safe incremental checkpoints.
- [] Deterministic spatial/time matching that distinguishes route-wide alerts, proximity, administrative status, and uncertainty.
- [] Historical queries exclude later raw versions, geometry changes, extracted facts, and human reviews.
- [] Real-source 40-case evaluation, rules-only comparison, held-out results, and an error analysis. Extend collection if real material is insufficient.
- [] A working local open-weight model path, with schema/span validation, source-backed explanations, and a usable deterministic fallback when AI is unavailable.
- [] Replay/idempotency, partial-feed, outage, schema-change, missing-location, DST, and time-cutoff scenarios have observed results.
- [] A clean-start demo with map/timeline, changes, evidence, uncertainty, and freshness; no unsupported causal/safety statements.
- [] An open-source code release package with dependencies, model license, data attribution, source-specific redistribution rules, runbook, case study, and measured results.

A natural correction or overlap may not occur during a short collection window. The completion gate requires functioning history/matching and honest live evidence; a claim that you found a real correction/overlap requires an observed case. Continue collecting rather than fake the finding.

8. Reuse from the core course

Existing work	Reuse here
R-P5 document processing	Structured extraction, validators, Airflow recovery, review workflow
R-P7 evaluation harness	Versioned cases, baseline comparisons, case-level reports
R-P8 prompt versioning	Record the exact extraction/explanation prompt used
R-P9 gateway	Local model access, request limits, usage/runtime accounting
R-P10 guardrails	Strip unnecessary sensitive/address data from model inputs and logs
R-P12 case study	Architecture decisions, measured outcomes, failed approaches, operating notes

Your new learning is **live-source acquisition, incremental/replay-safe DE, PostGIS, temporal knowledge modelling, and constructing an original derived data product**. Retrieval by SQL and provenance replaces a generic vector-search chatbot.

First action when you start

On D61, verify terms and acquire tiny real samples from all three sources. Inspect category/coordinate/alert coverage before choosing the corridor. Start your collector on D62; the dataset becomes yours through the collection history and derived modelling, not through downloading someone else's completed analysis.